Large Language Models: A Hands-on Approach

Distributed Training

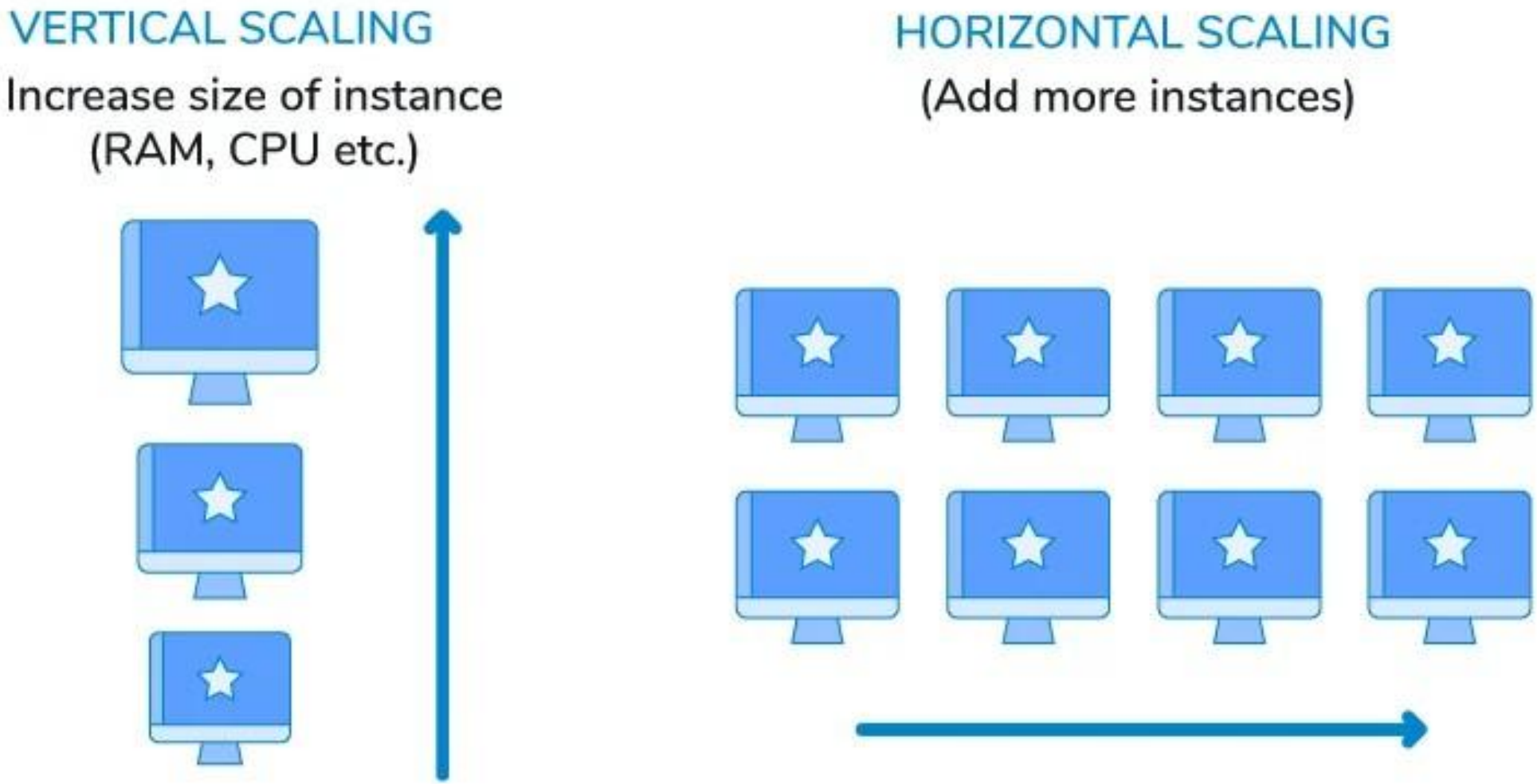
Distributed Training

The dataset is quite big, because it is made up of millions of articles, each of them with thousands of tokens. To train this model on a single GPU may be possible, but it poses some challenges

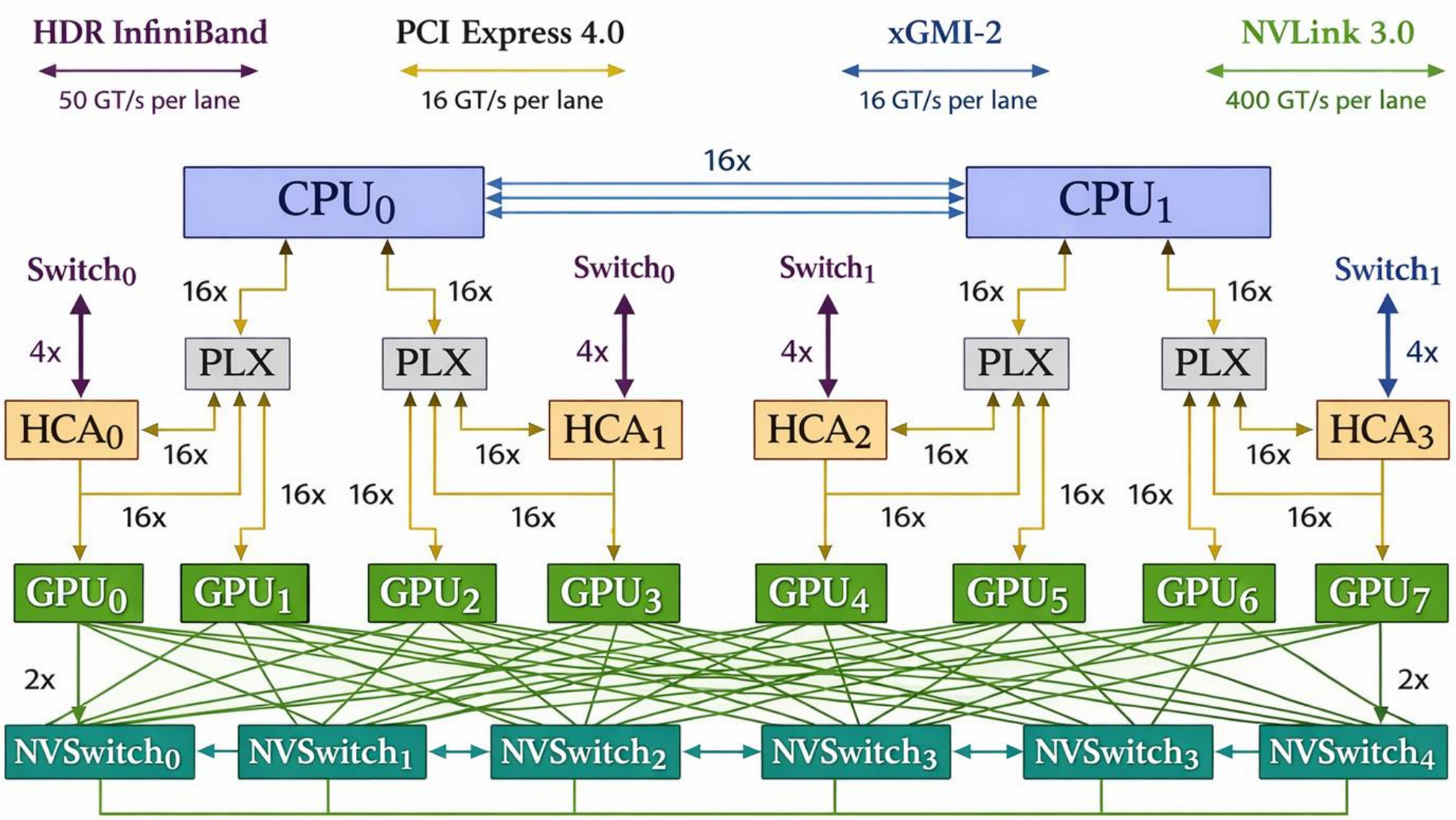
1. The model may not fit on a single GPU: this happens when the model has many parameters.
2. You are forced to use a small batch size because a bigger batch size leads to an **Out of Memory** error on CUDA.
3. The model may take years to train because the dataset is huge.

If any of the above applies to you, then you need to scale your training setup. Scaling can be done **vertically**, or **horizontally**..

Distributed Training



Multi GPU system



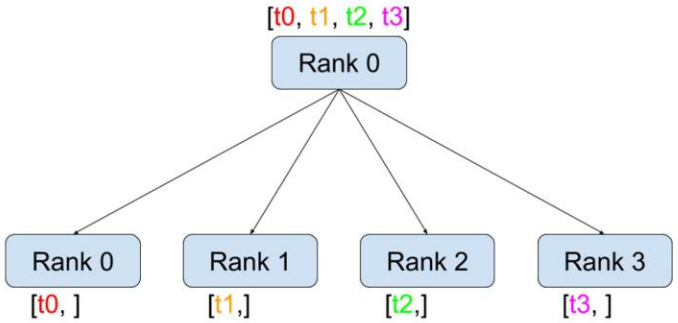
HCA :
Host Channel Adapter

PLX :PCIe Switch

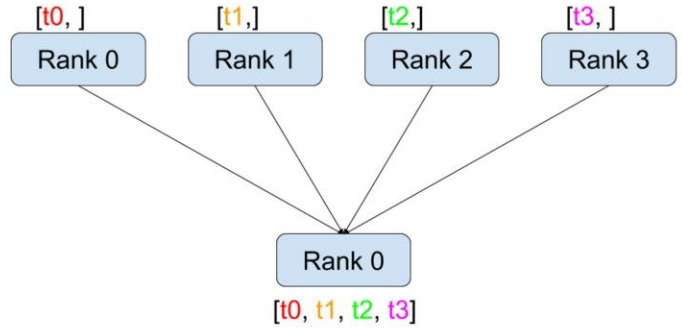
GT/s = Giga Transfers per second

Collective Communication

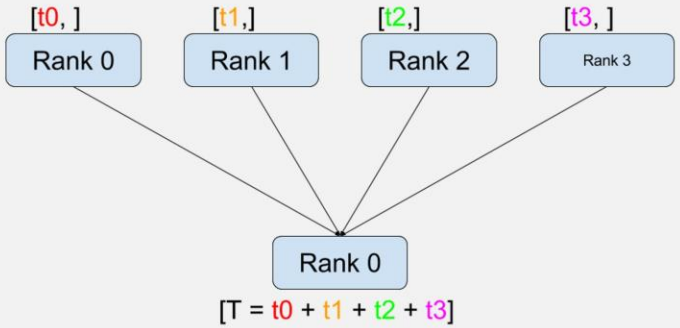
Scatter



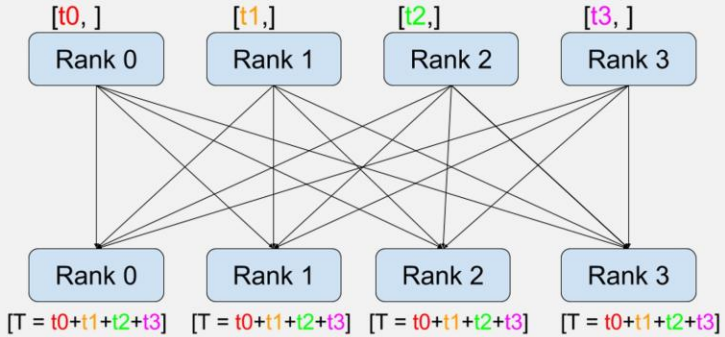
Gather



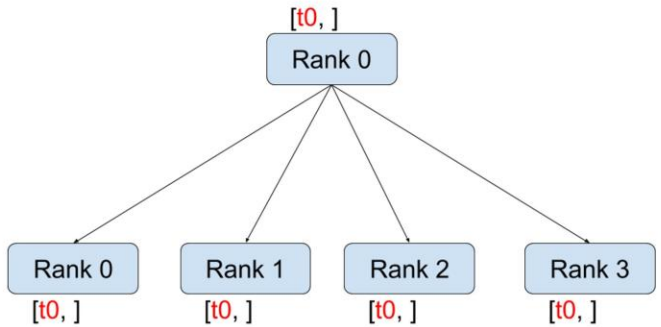
Reduce



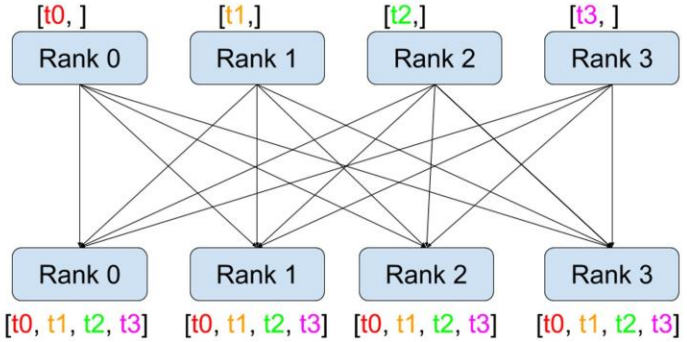
All-Reduce



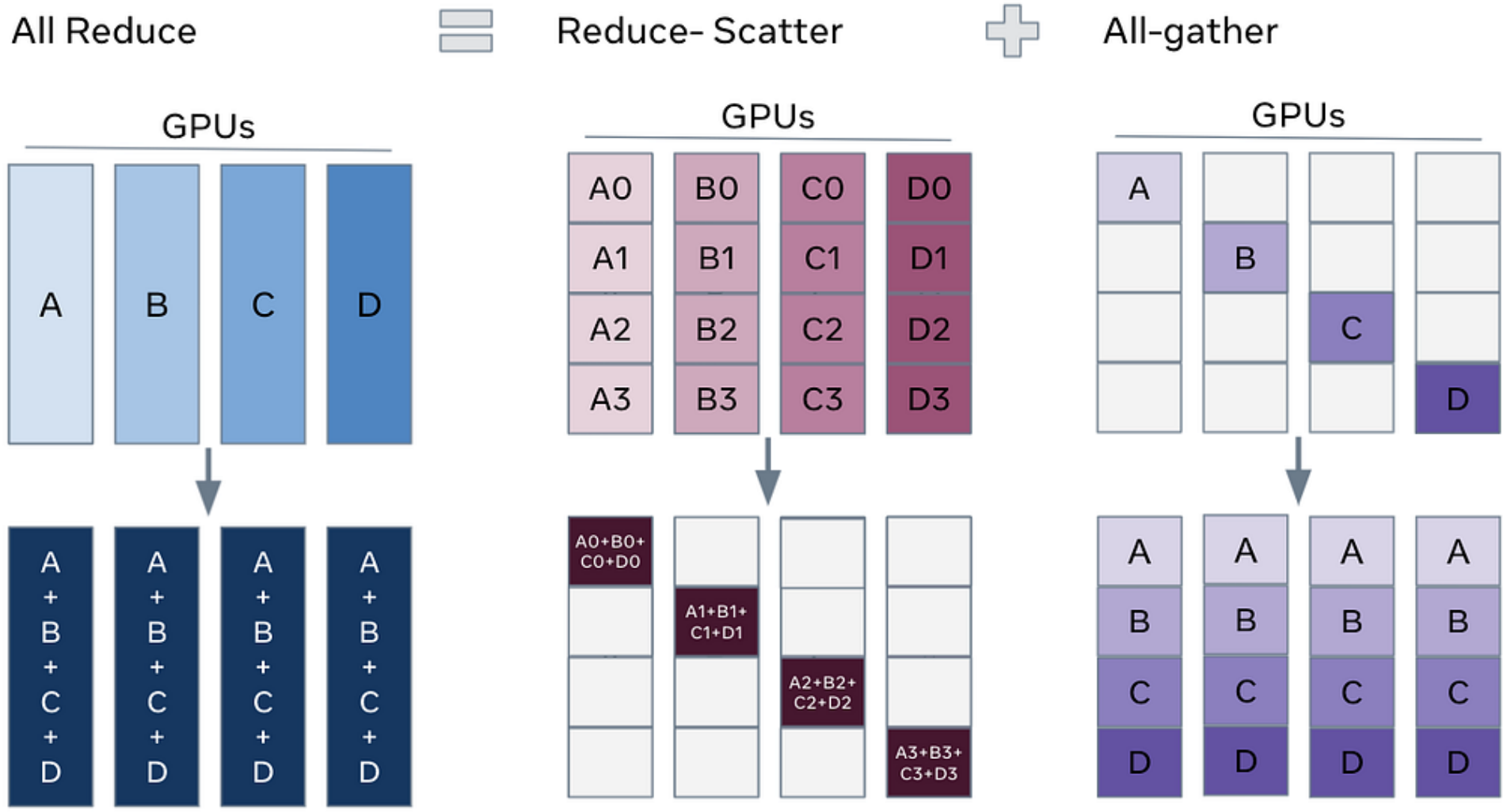
Broadcast



All-Gather

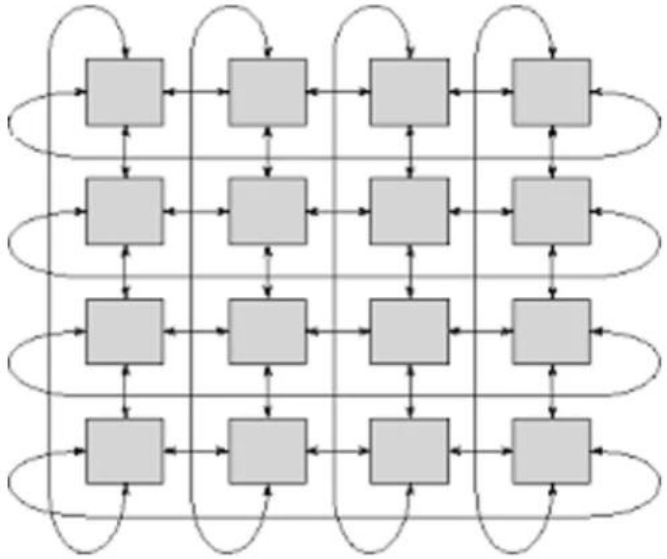


Collective Communication

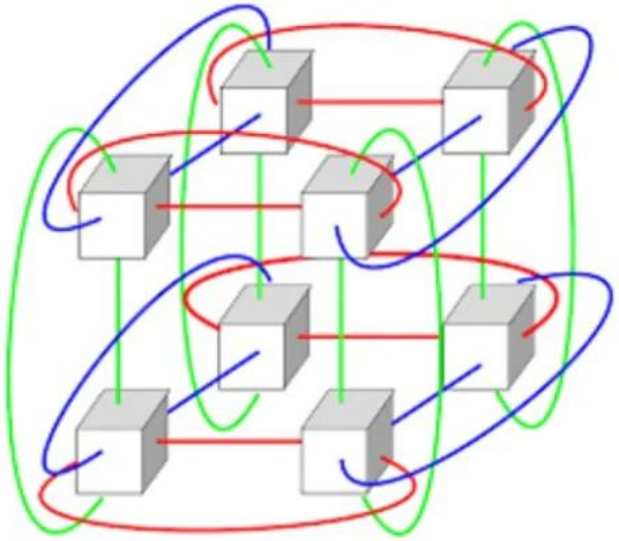


Different communication frameworks : GLOO, MPI, and NCCL

Communication Network

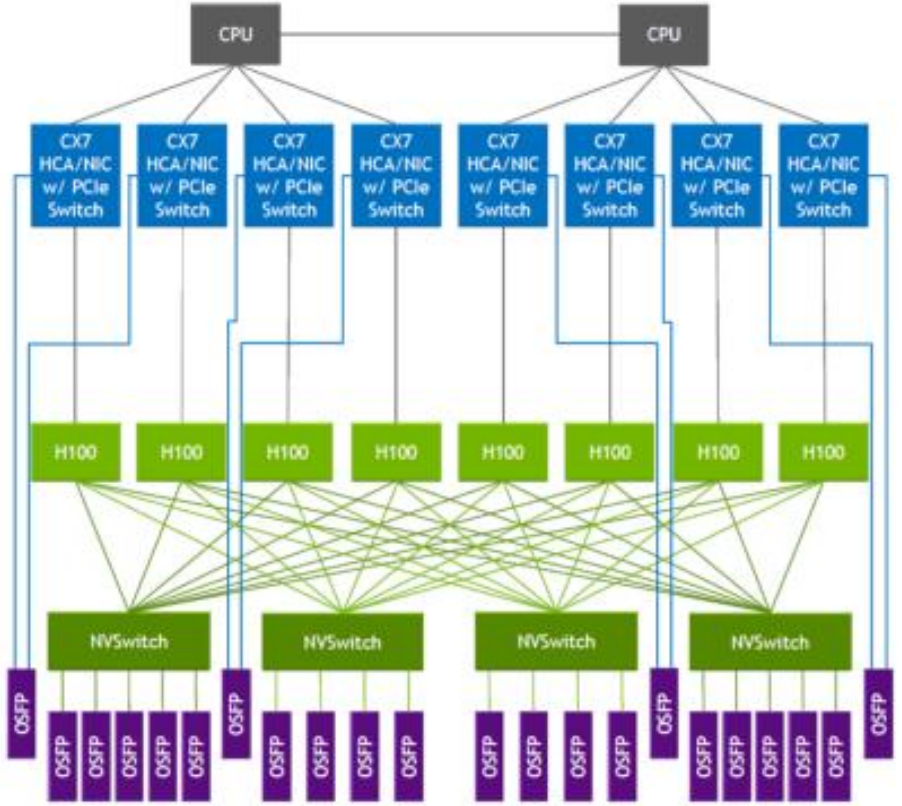


2D



3D

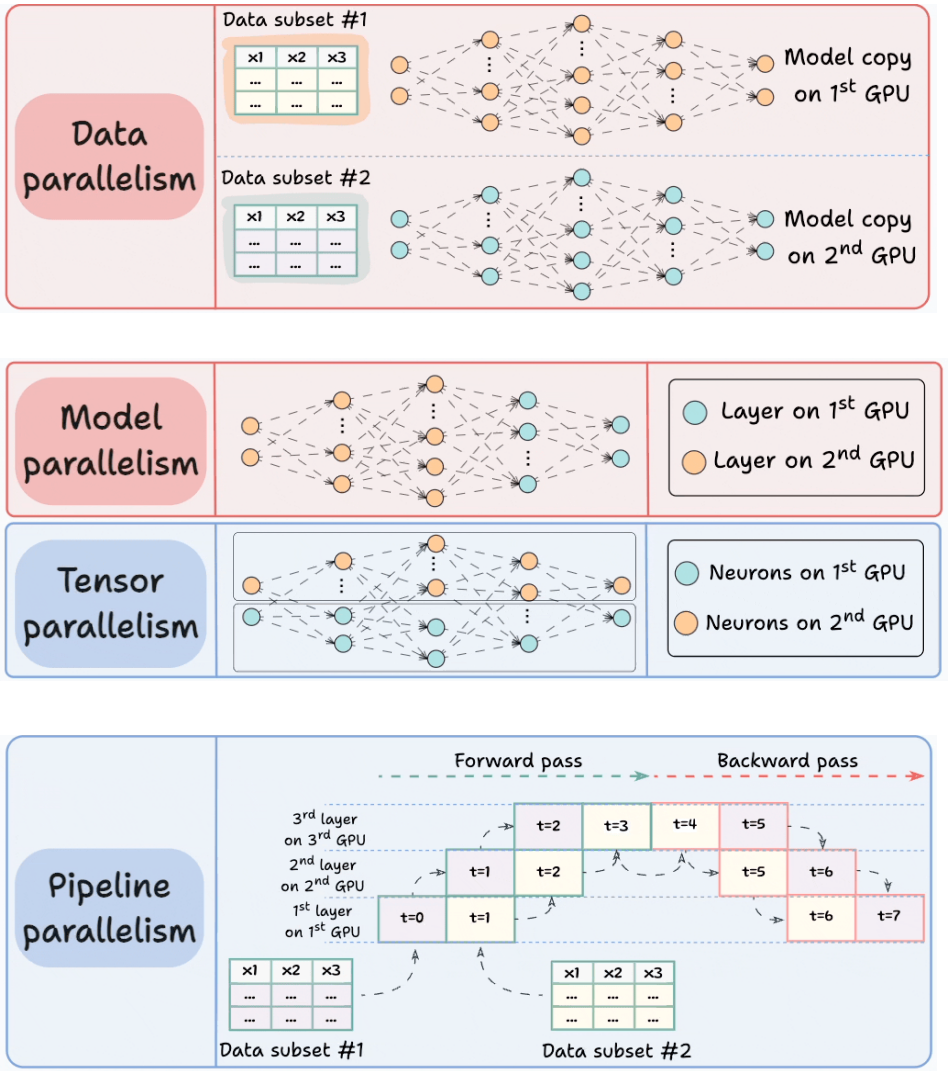
TPU



GPU

Standard LLM Distributed Parallelization

- **Data parallelism**
  - Naïve data parallel
  - ZeRO stage 1–3
- **Model parallelism**
  - Tensor parallel
  - Pipeline parallel
- **Activation parallelism**
  - Sequence parallel



Benefits of Distributed Training

1. **Fault Tolerance and Reliability**
   Distributed systems are **more resilient to failures** compared to a single machine.

2. **Efficiency**
   Distributed systems improve **training speed** by dividing the workload.

3. **Scalability**
   Distributed systems are **naturally scalable**.
   If more computational power is needed, we can simply **add more machines**

4. **Cost Effectiveness**
   Distributed systems can be **more cost-efficient** compared to large centralized systems.

Challenges of Distributed Training

1. **Data Parallelism Limitation :**
   Data parallelism does not reduce the memory footprint per device.
   - Each GPU still needs a **full copy of the model**.
   - Very large models cannot fit into GPU memory.

   Example:A model with more than 1 billion parameters may run out of memory even on GPUs with 32 GB memory.

2. **Model Parallelism Limitation**
   Model parallelism does not scale efficiently beyond a single node.
   - **Fine-grained computation**
   - **Expensive GPU communication**
   - **Frequent synchronization**

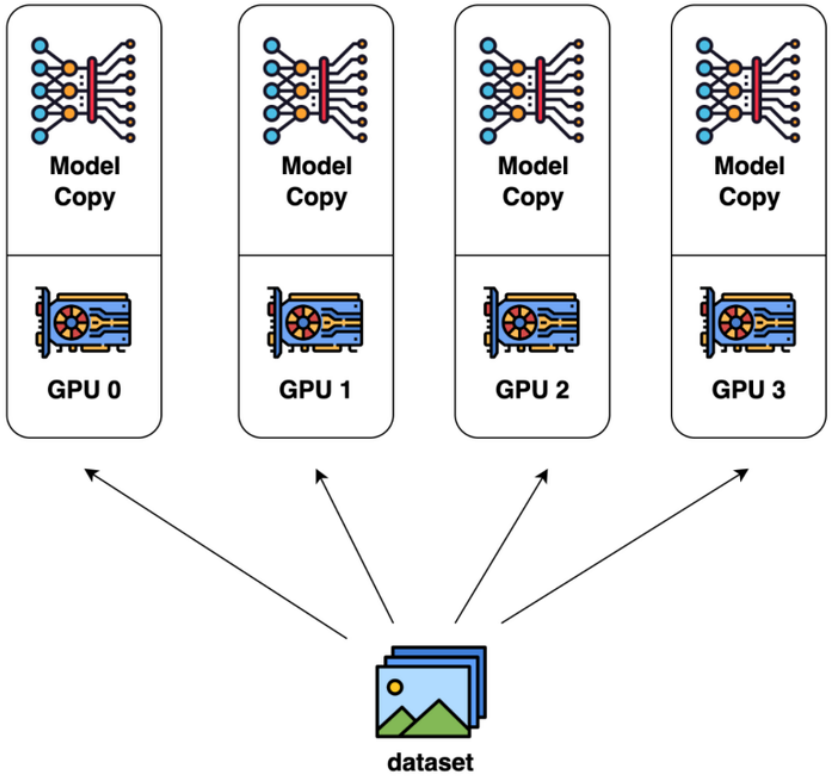
   Additionally, model parallelism frameworks often require:
   - **Extensive code modifications**
   - **Model architecture–specific implementations**

Naïve data parallelism

- **Naive parallelism:** split the elements of B sized batch across M machines.
- **How does this do?**
  - Compute scaling – each GPU gets B/M examples.
  - Memory scaling – none. Every GPU needs # params at least
  - Communication overhead – transmits 2x # params every batch

Gradient synchronization → #params
Parameter sharing → #params

Total communication → 2 × #params



Memory seems like to be a problem –
we copy the model parameters to each GPU.

What is in the GPU memory?

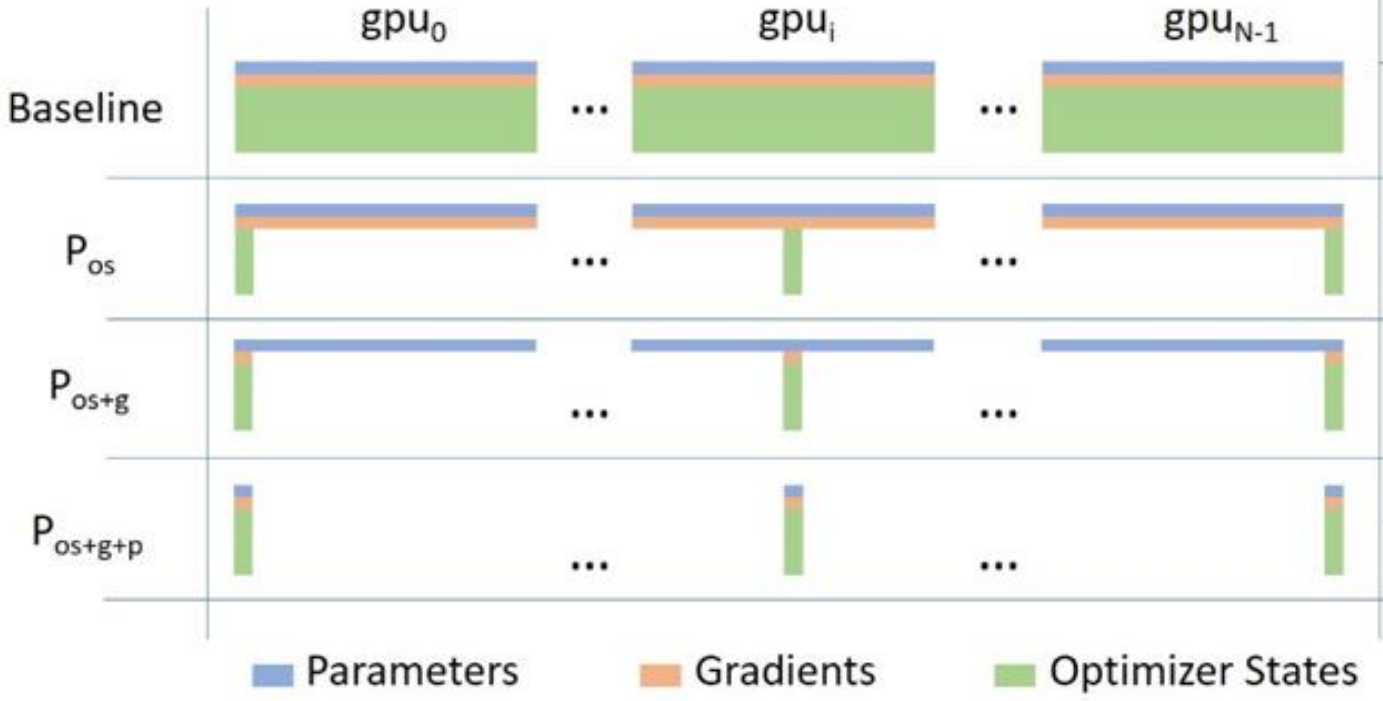
Assuming FP16 and x parameters (all trainable)

model states (or *training states*):

- **Parameters ⇒ 2x** *(2 bytes for float16)*
- **Gradients ⇒ 2x**
- **Optimizer State [Adam] ⇒ 12x**
  - Parameter copy **4x** *(4 bytes for float32)*
  - Momentum **4x**
  - Variance **4x**
  - **All stored in float32!!**

ZeRO : Solving the memory overhead issue of DP

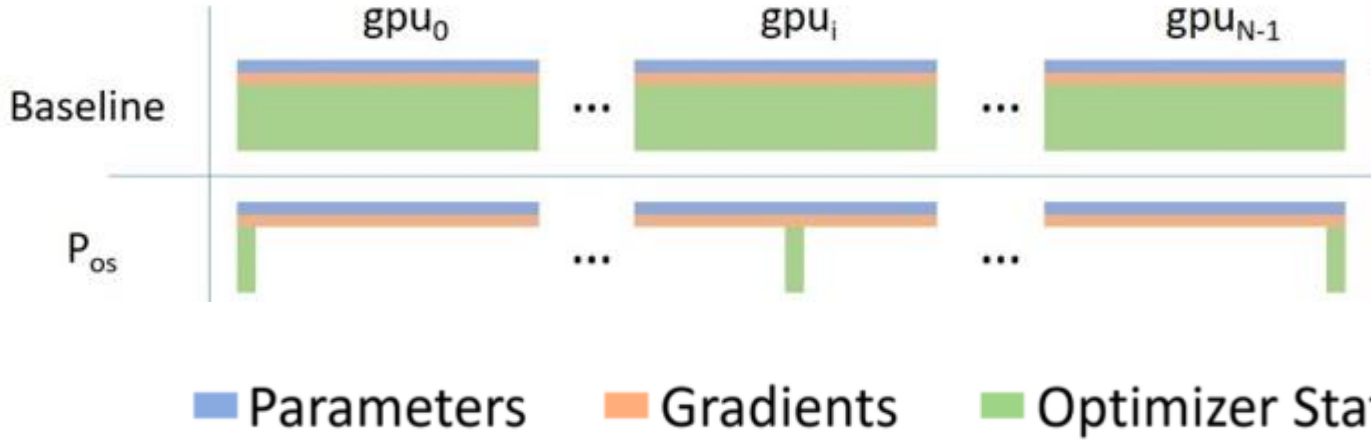
Core idea: split up the expensive parts (state) and use the reduce-scatter equivalence.



	Memory Consumption		Comm Volume
	Formulation	Specific Example K=12 Ψ=7.5B N_d=64	
Baseline	$(2+2+K)*\Psi$	120GB	1x
P_{os}	$2\Psi+2\Psi+\frac{K*\Psi}{N_d}$	31.4GB	1x
P_{os+g}	$2\Psi+\frac{(2+K)*\Psi}{N_d}$	16.6GB	1x
P_{os+g+p}	$\frac{(2+2+K)*\Psi}{N_d}$	1.9GB	1.5x

K → optimizer states per parameter, **Ψ (Psi)** → total number of model parameters,
$\mathbf{N_d}$ → number of GPUs in data parallel training

ZeRO stage 1. optimizer state sharding



	Memory Consumption		Comm Volume
	Formulation	Specific Example K=12 Ψ=7.5B N_d=64	
Baseline	$(2+2+K)*\Psi$	120GB	1x
P_{os}	$2\Psi+2\Psi+\frac{K*\Psi}{N_d}$	31.4GB	1x

High level idea:

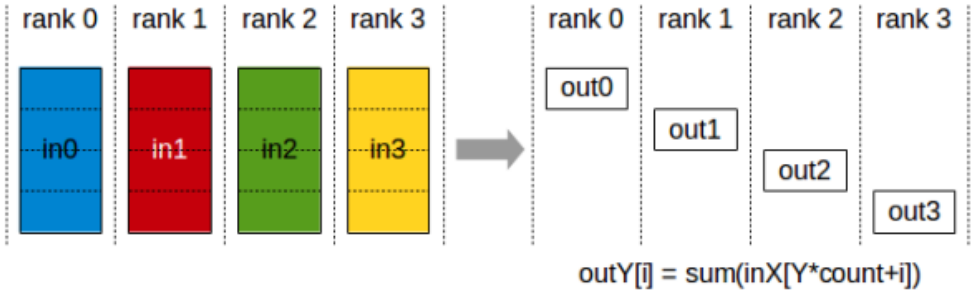
- Split up the optimizer state (first + second moments) across GPUs
- Everyone has the parameters + gradients

Each worker is responsible for updating a subset of params (corresponding to their slice)

ZeRO stage 1. how it works

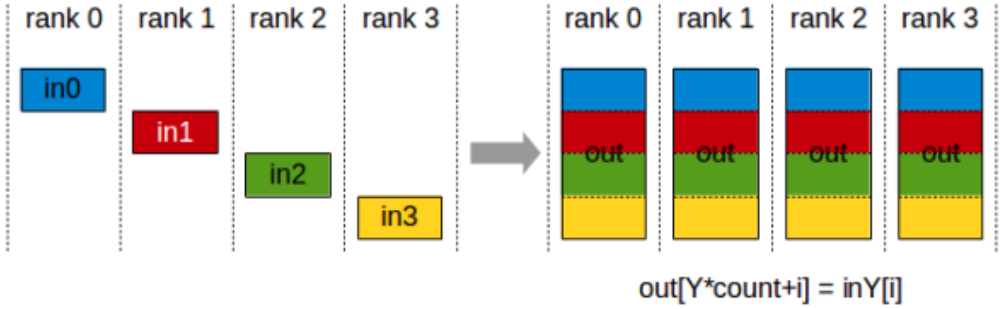
Step 1. Everyone computes a full gradient on their subset of the batch

Step 2. ReduceScatter the gradients – incur #params communication cost



Step 3. Each machine updates their param using their gradient + state.

Step 4. All Gather the parameters – incur #params communication cost



ZeRO stage 2. the simple extension to gradient sharding

Emboldened by our success, let's shard even more stuff

High level idea

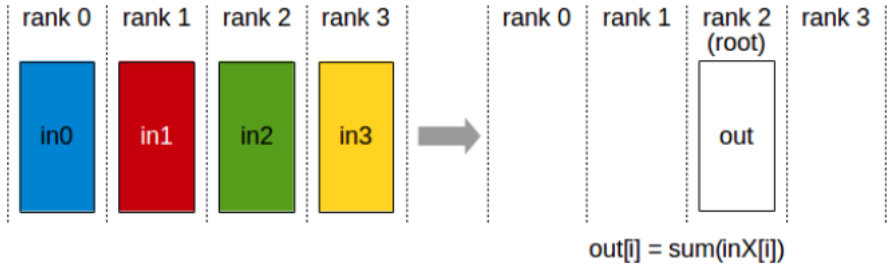
- Also keep the gradients (pink slices) sharded across the machines.
- Use the same (rough) tricks as stage 1.

Complexity – in backward pass we can never create and store in memory a full gradient vector, but each worker must compute a full gradient (since we're data parallel)

ZeRO stage 2. how it works

Step 1. Everyone incrementally goes backward on the computation graph

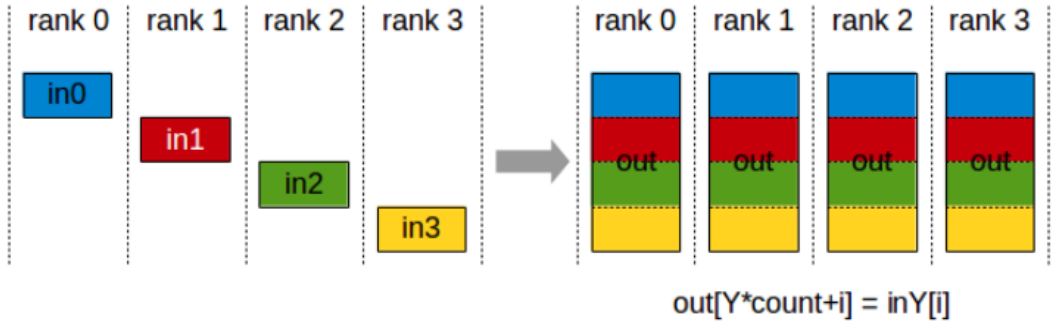
Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker



Step 1b. Once gradients are not needed in the backward graph, immediately free it.

Step 2. Each machine updates their param using their gradient + state.

Step 3. All Gather the parameters.



ZeRO stage 3 (aka FSDP) shard everything

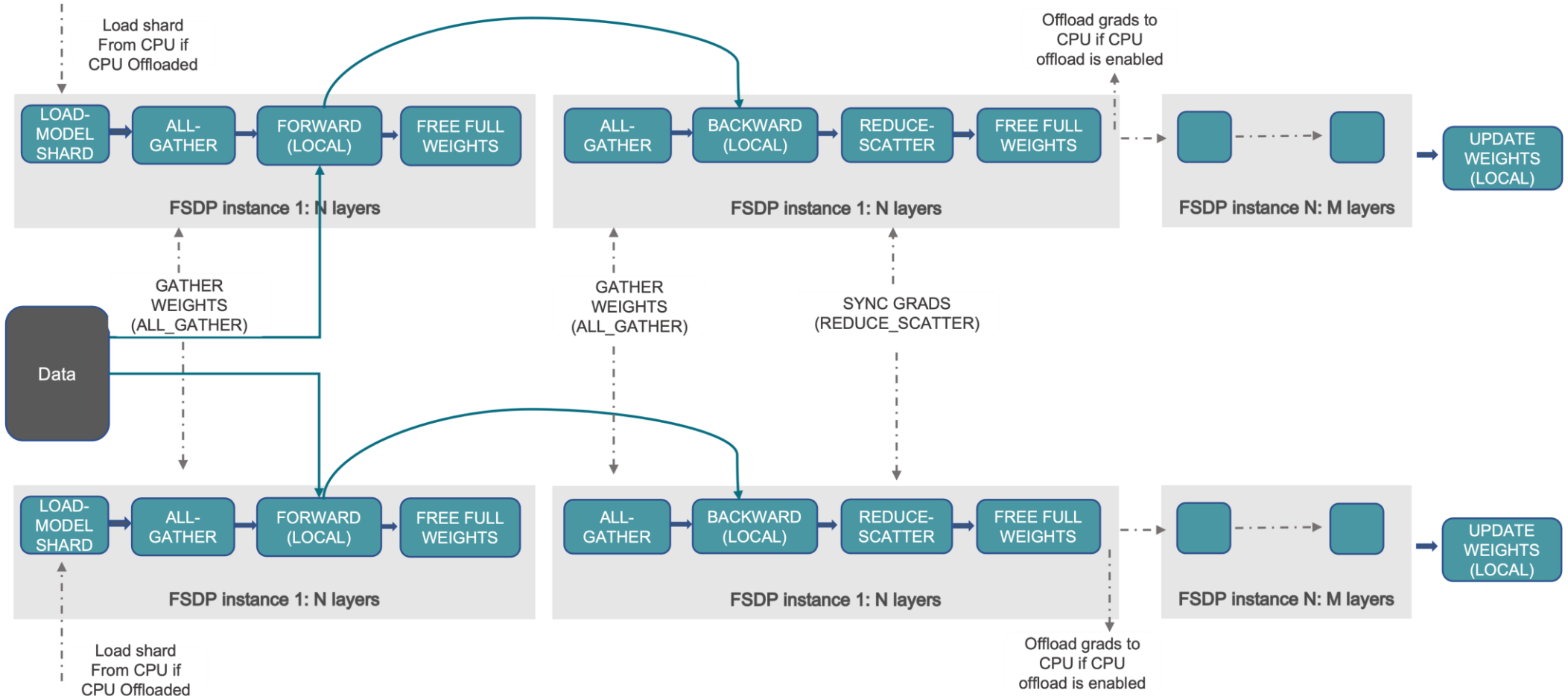
Fully Sharded Data Parallel (FSDP)

High level idea

- Sharding [Horizontal “Splitting”] everything – incl parameters!
- Use the same ‘incremental communication / computation’ ideas
- Send and request parameters on demand while stepping through the compute graph.

Is it possible to do this with low overhead?

ZeRO stage 3 (aka FSDP)



How FDSP / ZeRO stage 3 works

Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

Overlapping communication and computation ($W_1W_0 + W_2W_0)x = y$

- The all-gathers happen all at once while forward happens, masking the comm cost.

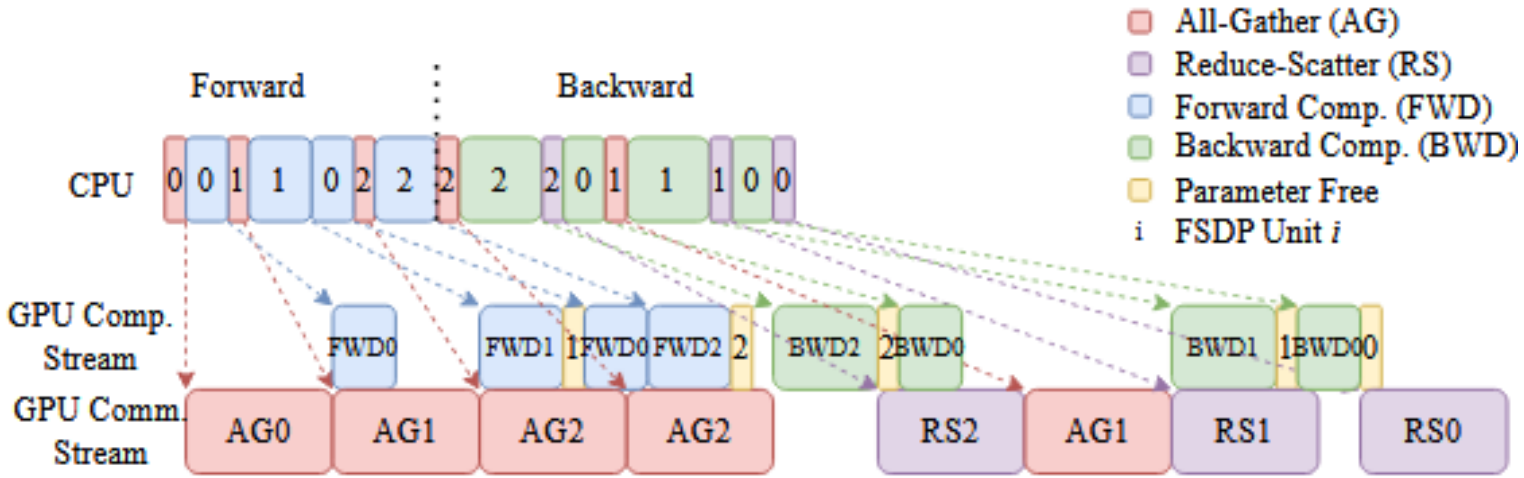


Figure 5: Overlap Communication and Computation

https://arxiv.org/pdf/2304.11277

Beyond data parallel – model parallelism

Scaling up in memory (without changing batch size) with model parallelism

What model parallelism is..

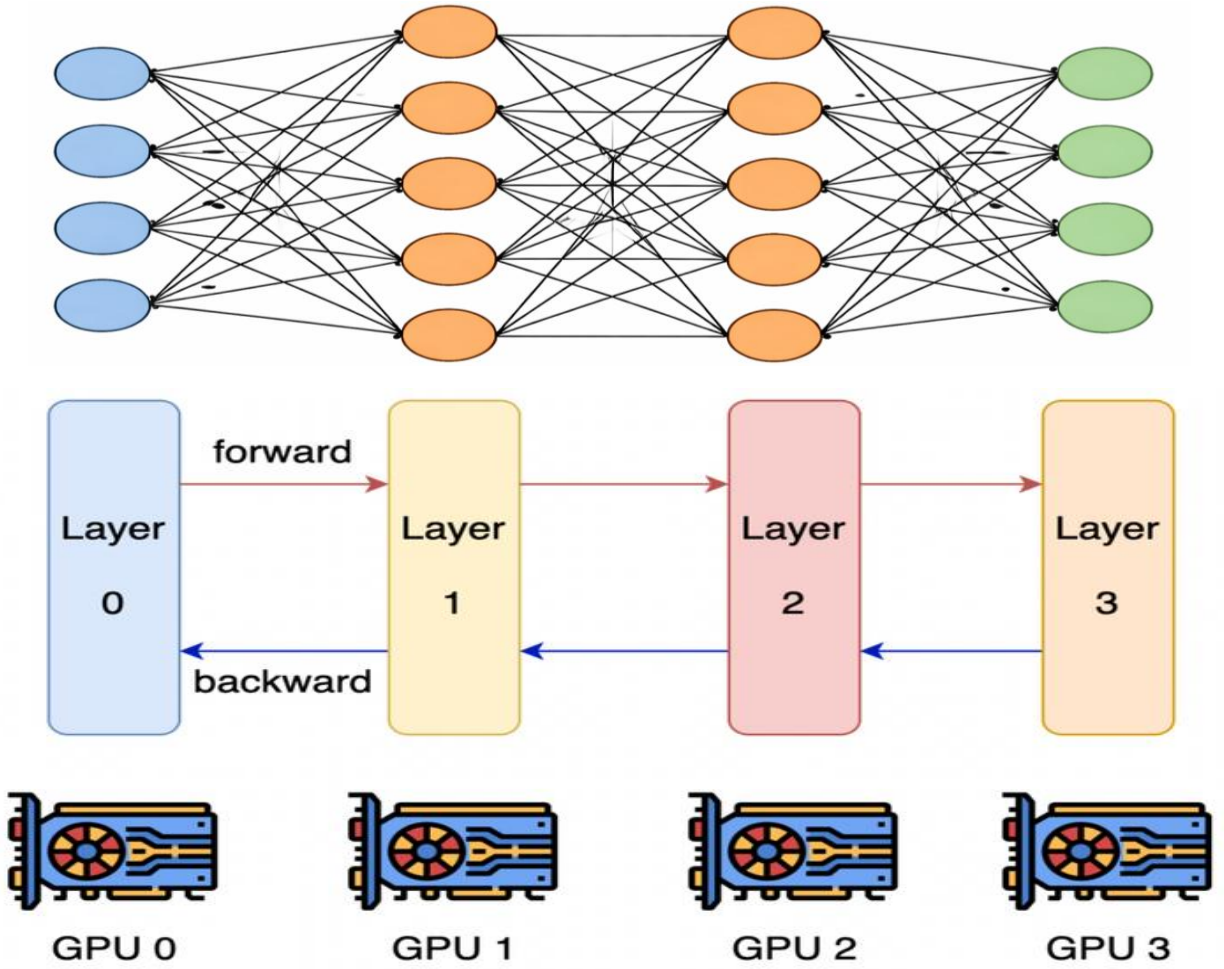
- It splits up the parameters across GPUs (like zero3)..
- But communicate activations (while zero3 sends params).

We cover two different types of parallelism

1. Pipeline parallel
2. Tensor parallel (+ Sequence parallel)

These correspond to two different ways of cutting up the model.

Layer-wise parallel



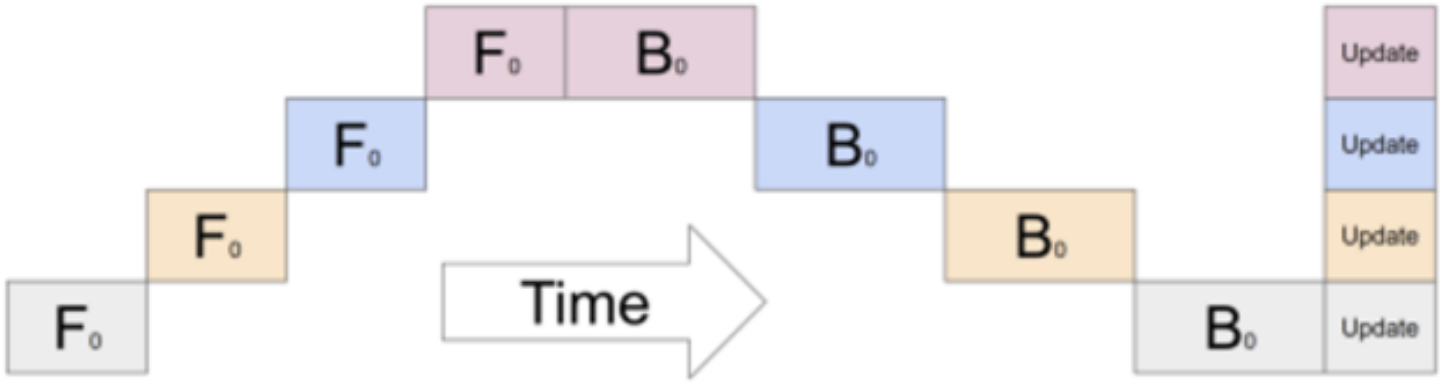
Layer-wise parallel cuts up layers, assigns some subset to GPUS.

Activations and partial gradients are passed back and forth

What's wrong with layer-wise parallel

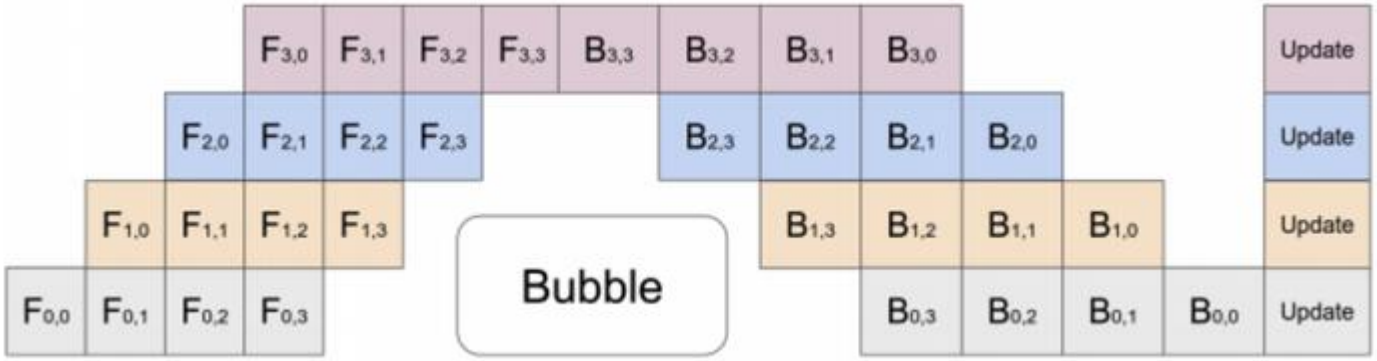
Utilization of layer-wise parallelism is *terrible..*

With n gpus, each gpu is active 1/n of the time.



Each GPU is idling most of the time, waiting for the backward pass to propagate back

A solution: pipeline parallel



Solution: Pipeline-parallel.

Process ‘micro-batches’ (in this case, 4).

Send off the first microbatch and start computing the second.

The ratio of bubble time to useful compute is . $\frac{n_{stages}-1}{n_{micro}}$ so we need a big batch size!

Why pipeline parallel?

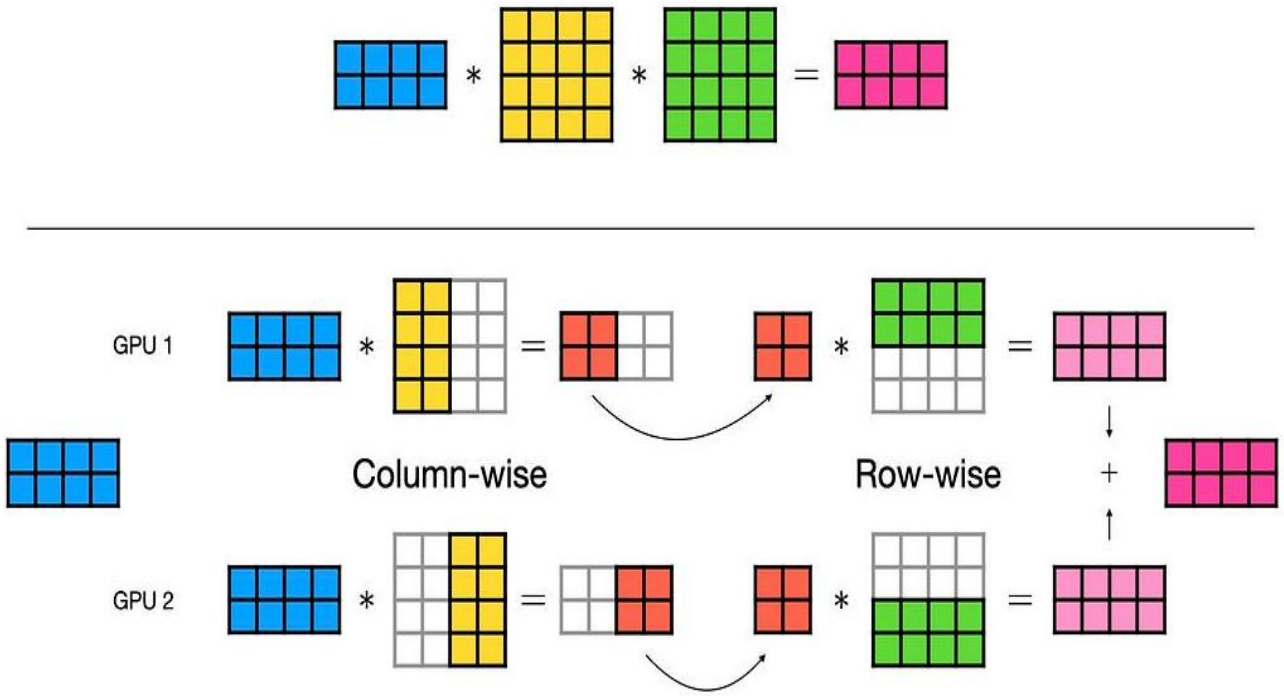
Pipelines seem terrible. Why do we do it?

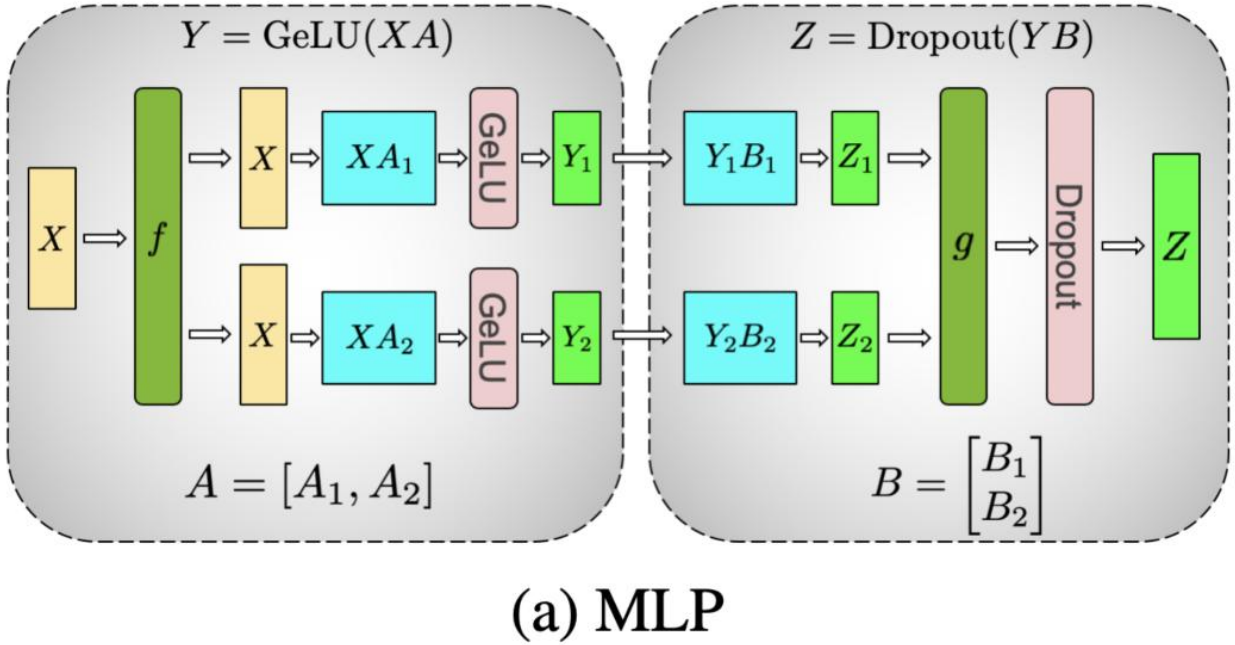
1. Pipelines save memory (compared to DDP)

2. Pipelines can have good communication properties (compared to FDSP)
   - it depends only on activations ($b \times s \times h$) and is *point to point*
     b : batch size; s : sequence length; h : hidden size

Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

Tensor parallel- GPUs have submatrices





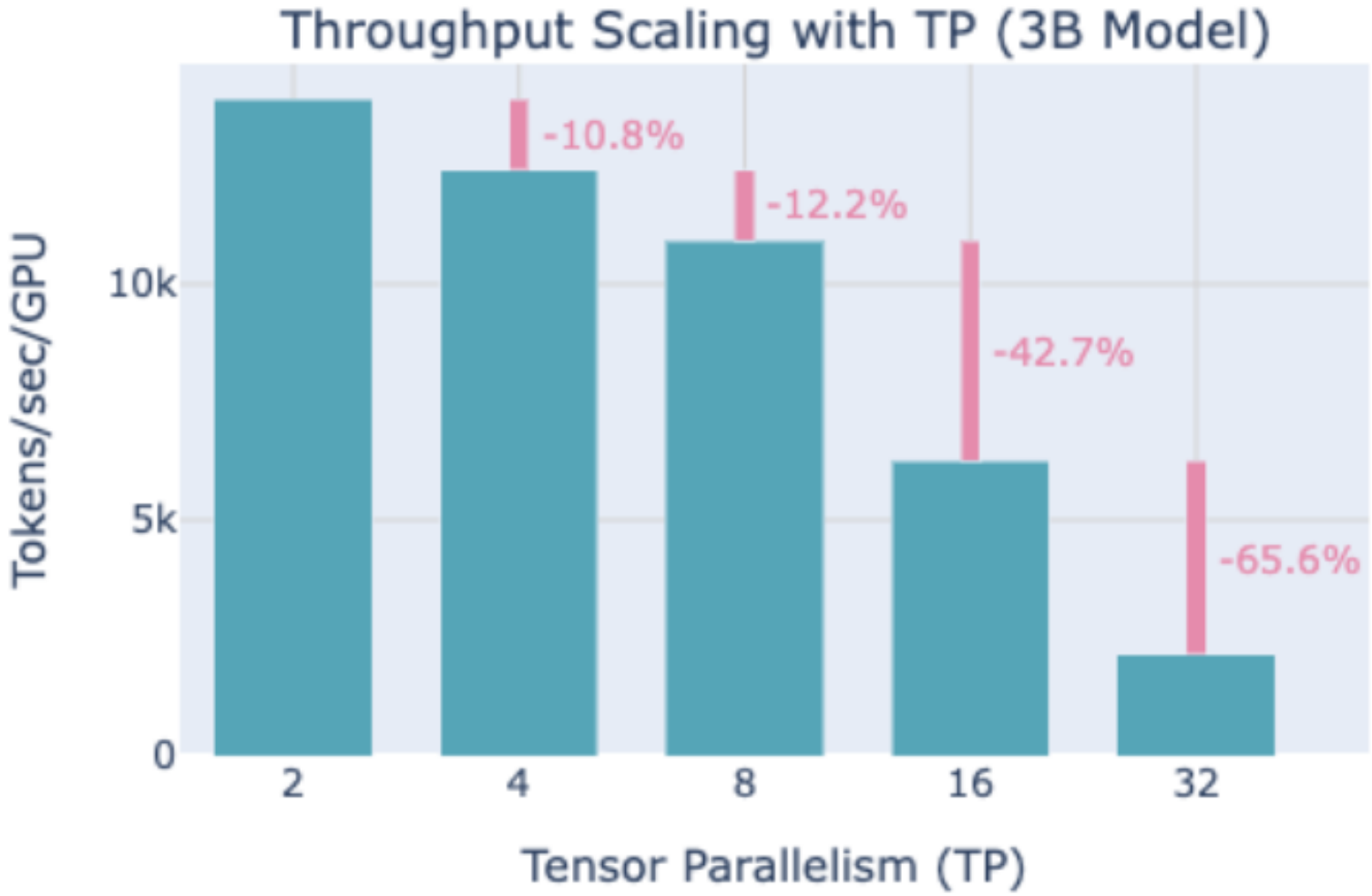
(a) MLP

Assign columns (A1, A2) and rows (B1, B2) to separate GPUs.

- In the forward pass, f is the identity, and g is an all-reduce.
- In the backward pass, f is an all-reduce, g is the identity.

https://arxiv.org/pdf/1909.08053

Tensor parallel why?



Tensor parallel – pros and cons vs pipeline parallel

How do things compare to pipeline parallel?

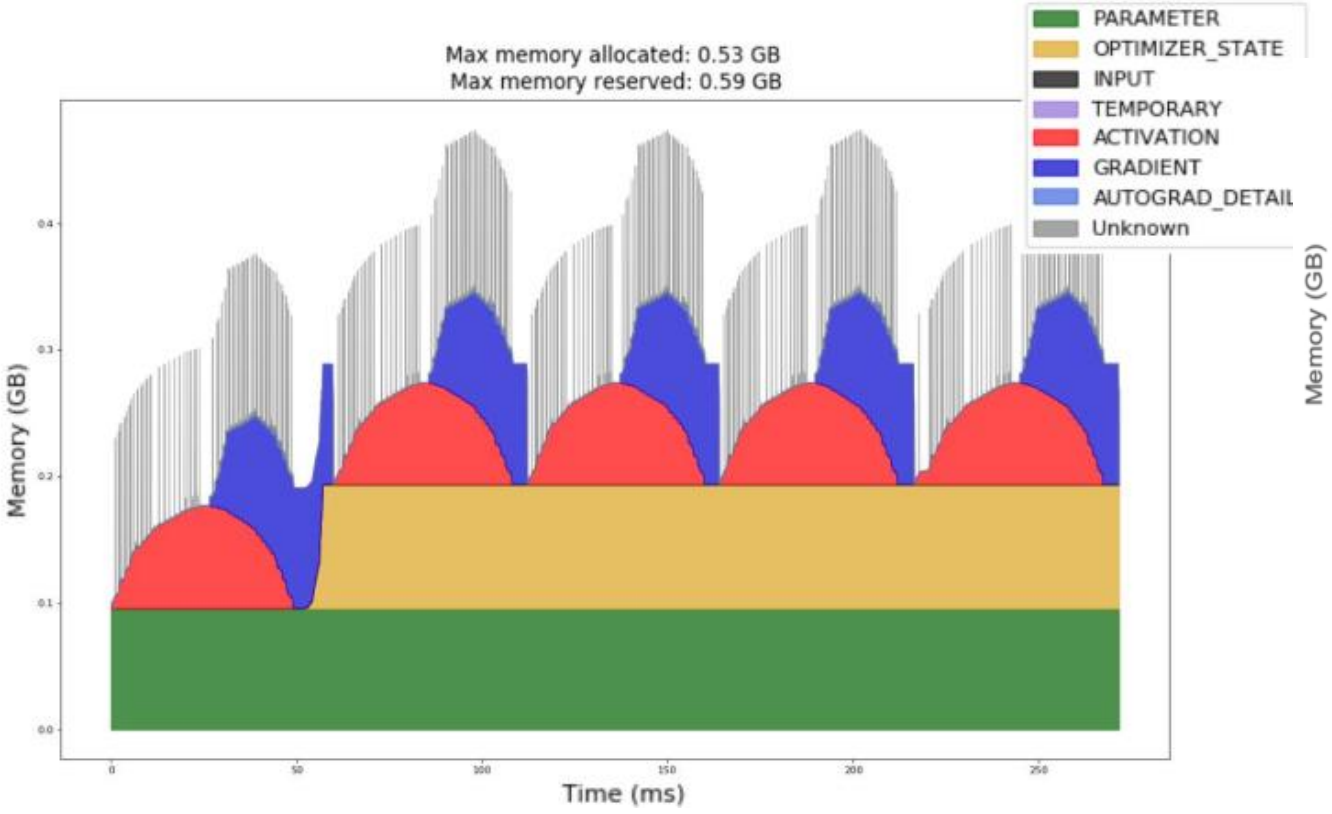
Pros – no bubble. If your network is fast enough, there's no waiting for others.
- low complexity – simple to 'wrap' models without major infra changes
- doesn't need large batch sizes to work well

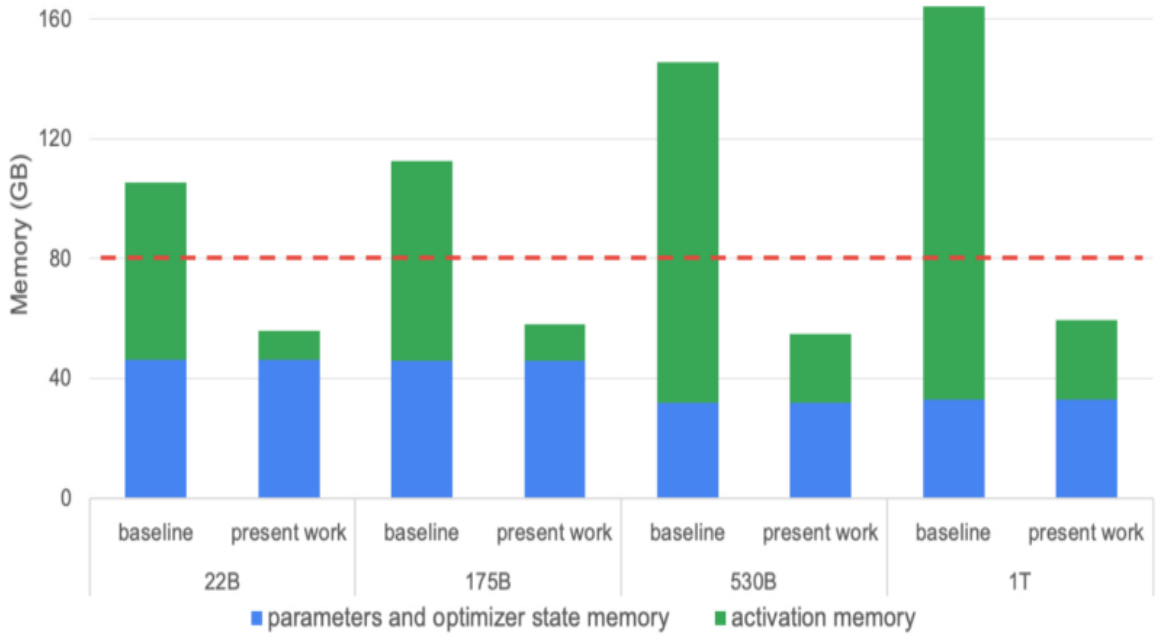
Cons – *much* larger communication than pipeline parallel.

- *Pipeline*: bsh point-to-point communication per microbatch
- *Tensor*: $8bsh\left(\frac{n_{devices}-1}{n_{devices}}\right)$ *per layer* and *all-reduce* communication.

Use tensor parallel whenever we have low-latency, high-bandwidth interconnects

A final complexity – activation memory





*https://arxiv.org/abs/2205.05198

Sequence parallel

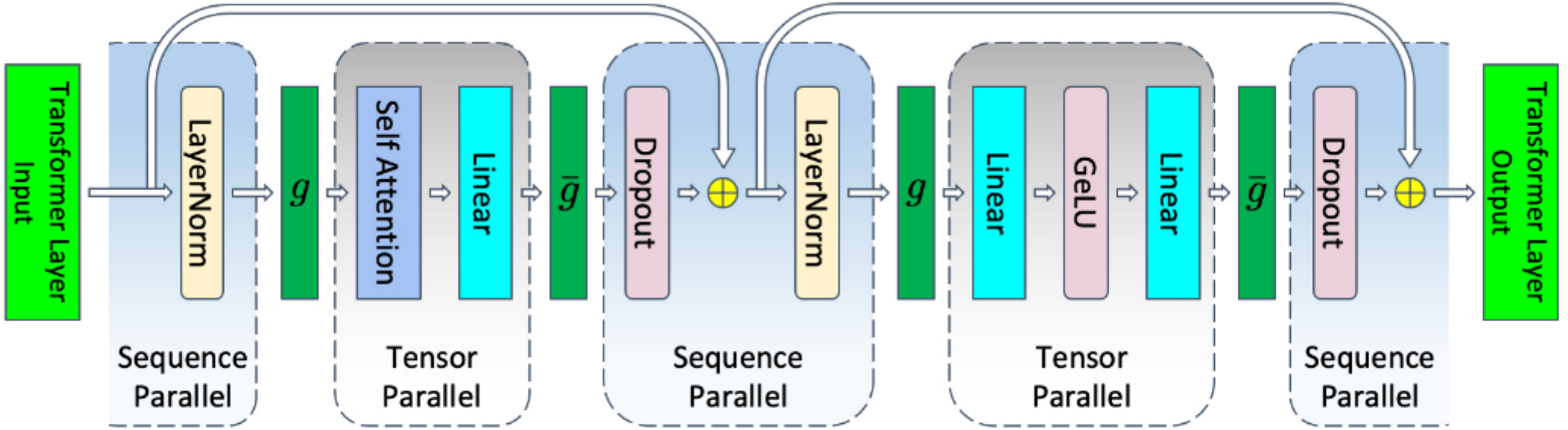


Figure 5. Transformer layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in the forward pass and reduce-scatter in the backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

Observation: all the 10bsh terms are pointwise ops over the sequence

… so split up the layer norm/dropout layers along the sequence axis.

- In the forward pass, 'g' is an all gather, '$\bar{g}$' is reduce-scatter
- In the backward pass, the two are reversed

*https://arxiv.org/abs/2205.05198

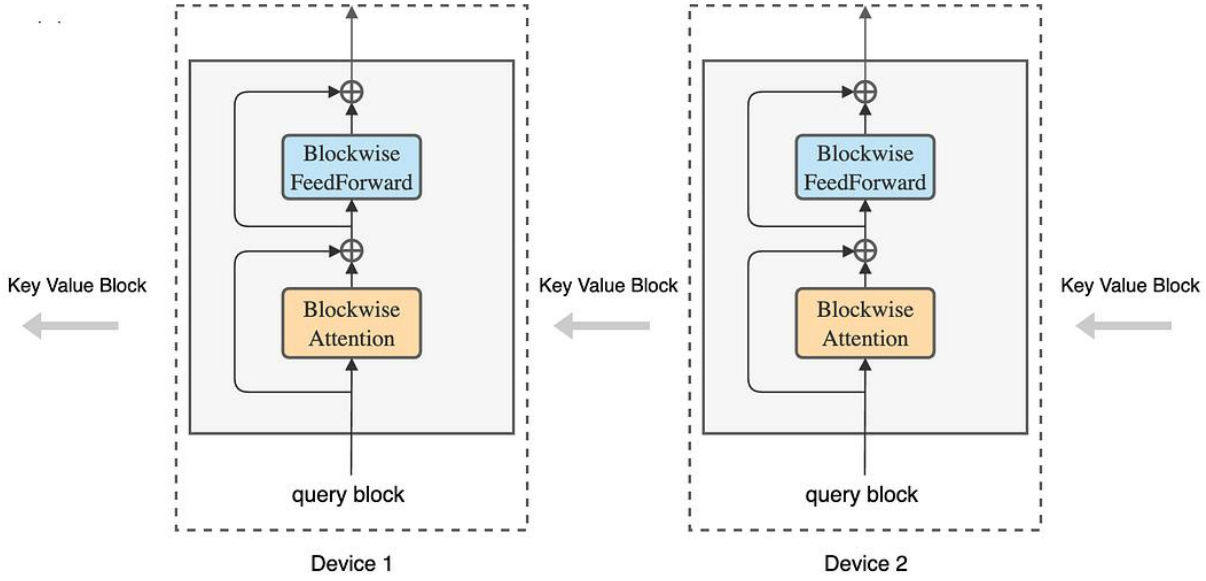
LLM parallelism

	Sync overhead	Memory	Bandwidth	Batch size	Easy to use?
DDP/ ZeRO1	Per-batch	No scaling	2* # param	Linear	Very
FSDP (ZeRO3)	3x Per-FSDP block	Linear	3 * # param	Linear	Very
Pipeline	Per-pipeline	Linear	Activations	Linear	NO
Tensor+ seq	2x transformer block	Linear	8*activations perlayer all-reduce	No impact	No

LLM parallelism

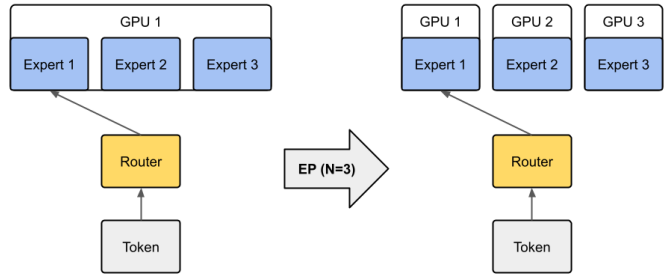
	Max Parameter (in billions)	Max Parallelism	Compute Efficiency	Usability (Model Rewrite)
Data Parallel (DP)	Approx. 1.2	>1000	Very Good	Great
Model Parallel (MP)	Approx. 20	Approx. 16	Good	Needs Model Rewrite
MP + DP	Approx. 20	> 1000	Good	Needs Model Rewrite
Pipeline Parallel (PP)	Approx. 100	Approx. 128	Very Good	Needs Model Rewrite
PP + DP	Approx. 100	> 1000	Very Good	Needs Model Rewrite
MP + PP + DP	> 1000	> 1000	Very Good	Needs Significant Model Rewrite
ZeRO	*> 1000*	*> 1000*	Very Good	Great

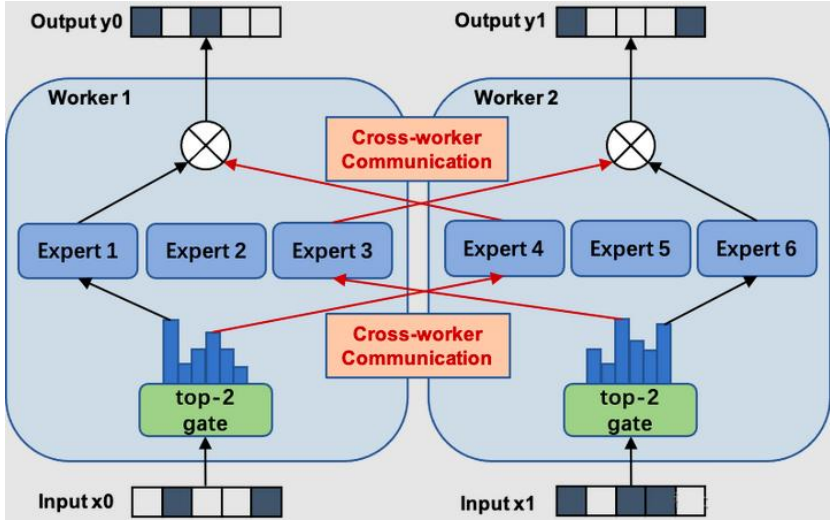
Other parallelism strategies



Context parallel / Ring attention
(split activations across GPUs in a long sequence)

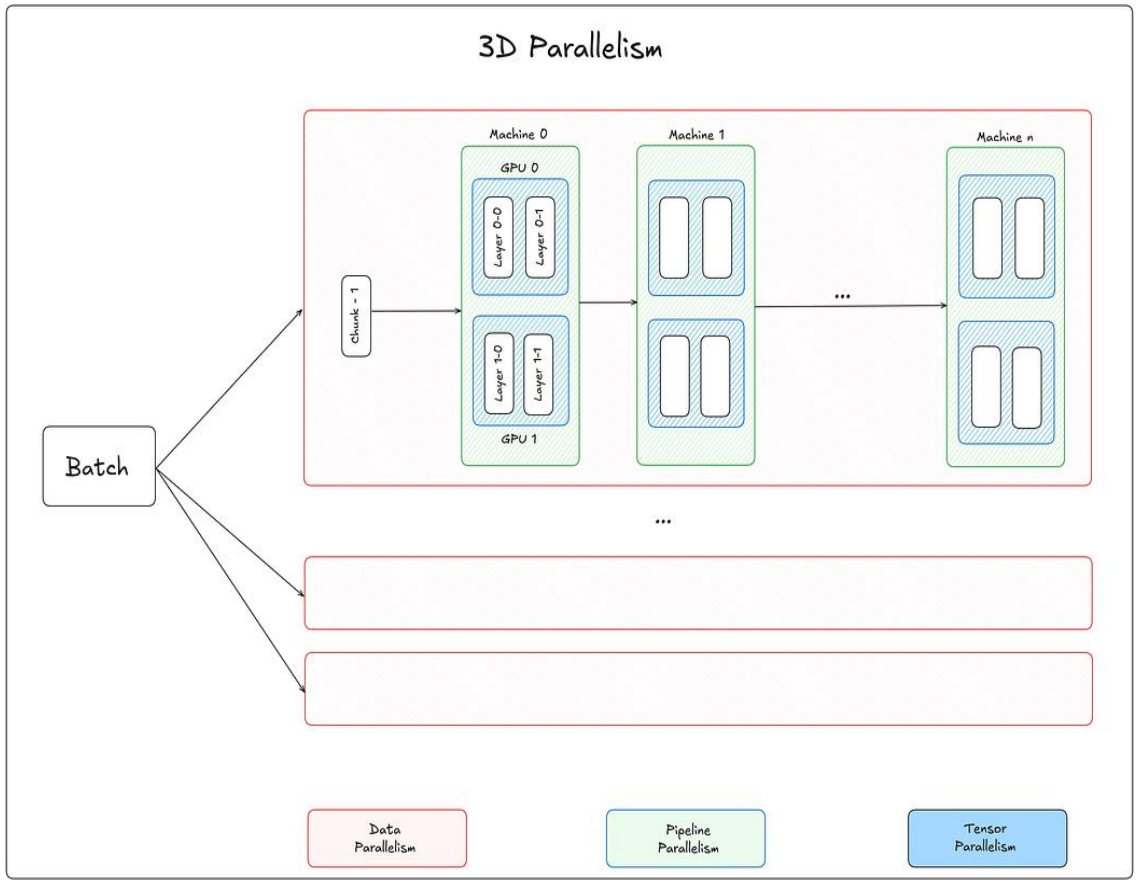
Expert Parallelism applied on Mixture-of-Experts.

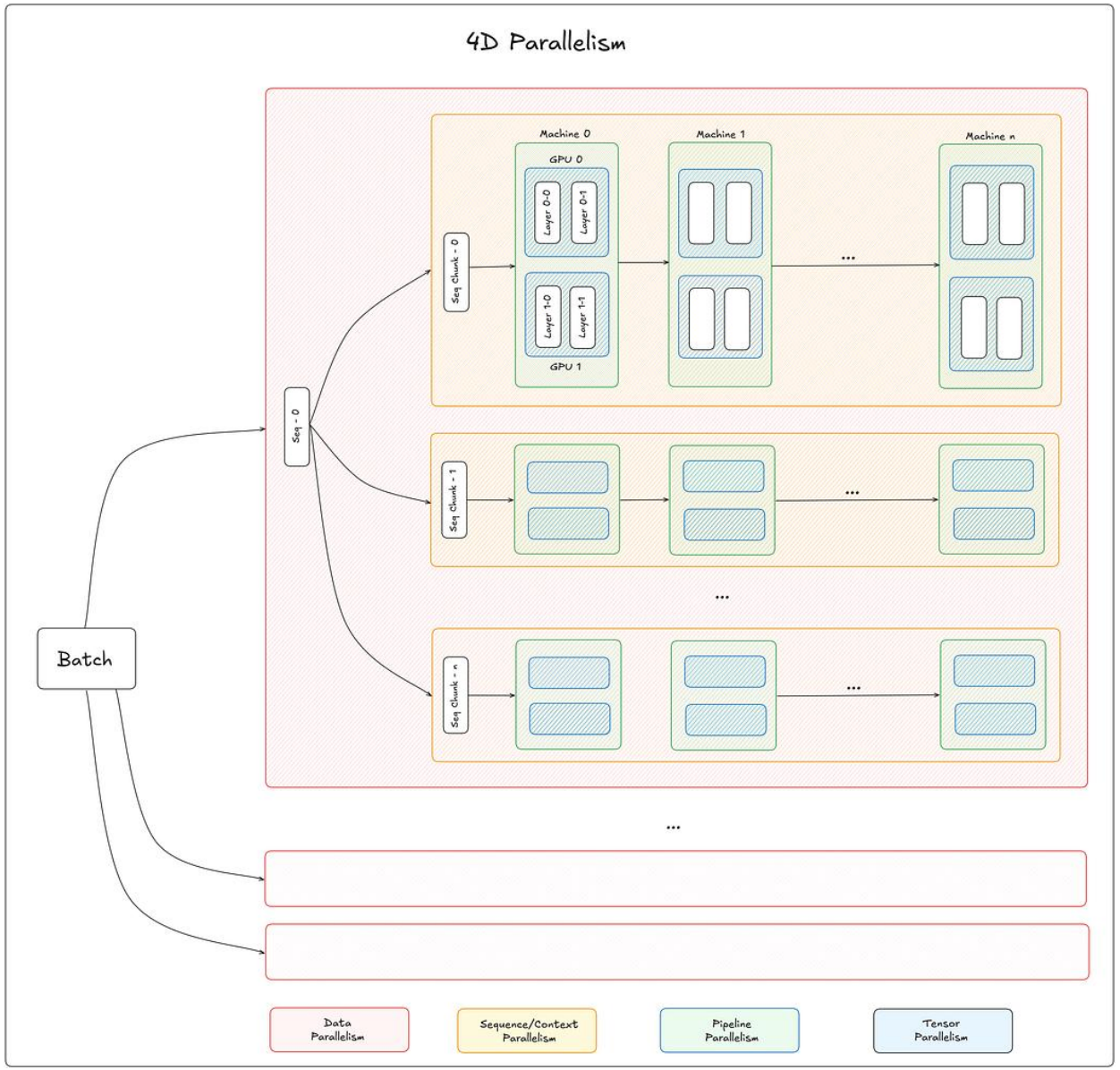




Expert parallel
(split experts across GPUs)

Other parallelism strategies





Thank You